

Enterprise Multi-Agent RAG System: Complete Case Study

Author: Vithun T R | Version: 1.0.0-PROD | Architecture: Fail-Closed Tri-Agent Engine

Part 1: Executive Overview & Problem Context

1. Executive Summary

This case study details the design, implementation, and empirical validation of an enterprise-grade Multi-Agent RAG system resolving hallucination, prompt injection, and latency degradation.

2. Problem Statement

Standard single-agent RAG pipelines suffer from prompt injection risks in untrusted context, unbounded hallucinations, model deprecation breaks, and high operational latency.

3. Project Vision & Business Value

Delivers zero-hallucination knowledge retrieval, deterministic refusal bounds, and sub-3.0s SLA compliance for mission-critical enterprise applications.

4. Target Architectural Scope

Features non-blocking similarity search, strict JSON validation guardrails, dynamic model fallback, and chunk-level traceability.

Part 2: System Architecture & Design Strategy

5. Multi-Agent Design Pattern

Employs a tri-agent orchestrator: Agent 1 (Vector Retriever), Agent 2 (Fail-Closed Security Evaluator), and Agent 3 (Grounded Synthesis Engine).

6. Agent 1 Spec (Vector Store Retriever)

ChromaDB dense vector similarity search operating with k=2 top-k chunk retrieval strategy.

7. Agent 2 Spec (Fail-Closed Security Evaluator)

Groq LPU QA Verifier enforcing structured JSON validation under strict data-instruction separation boundaries.

8. Agent 3 Spec (Grounded Synthesis Engine)

Grounded response synthesis engine constrained strictly to verified context blocks.

9. Component Interaction & Data Flow

Query Ingestion -> Chroma Retrieval -> JSON Gate Evaluation -> Decision Enforcement (PASS/REJECT) -> Grounded Synthesis.

10. Data Protection & Boundary Isolation

Separates system instructions from retrieved data using explicit structural tags.

Part 3: Security & Performance Analytics

16. Fail-Closed Gate Design Logic

Default REJECT initialization ensures security under parsing errors, API timeouts, or ungrounded queries.

17. Adversarial Threat Modeling

Defends against Indirect Prompt Injections embedded within vector stores.

20. Execution Latency Benchmarks

Valid Query: 2.25s (PASSED) | Adversarial Test: 1.05s (REJECTED) | Out-of-Scope: 0.98s (REJECTED).

Empirical Latency Matrix

Scenario	Retrieval	Guardrail	Synthesis	Total	Status
Valid Technical Query	0.12s	0.85s	1.28s	2.25s	PASSED
Adversarial Injection	0.11s	0.94s	Bypassed	1.05s	REJECTED